

Amortized Analysis: Further Examples

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Warm-Up: Enumerating Subsets

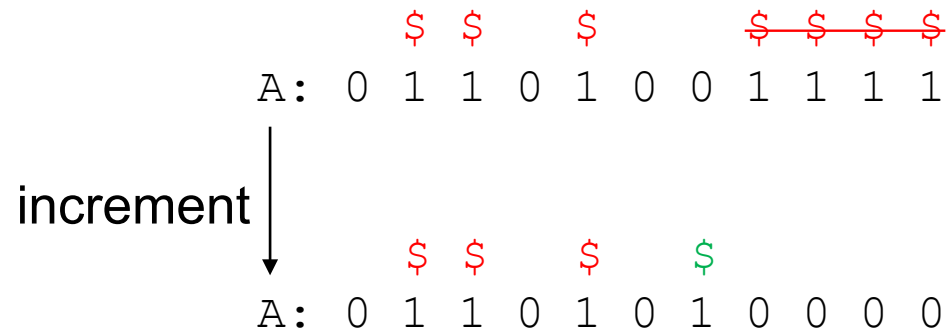
- How do you easily enumerate the 2^{20} different subsets of a set of 20 items?
- Use the bits of x while stepping x from 0 to $2^{20} - 1$:

$x = 00000000000000000000$

$x = 00000000000000000001$

$x = 00000000000000000010$

$x = 00000000000000000011$



Accounting: 1s carry credit to pay to switch themselves back to 0s

Potential: $\Phi = \#$ of 1s in counter.
The amortized cost of an increment (say, switching b 1s to 0s) is
actual cost + $(\Delta\Phi)$
 $b+1 + (-b+1) = 2$

Recursive Slowdown

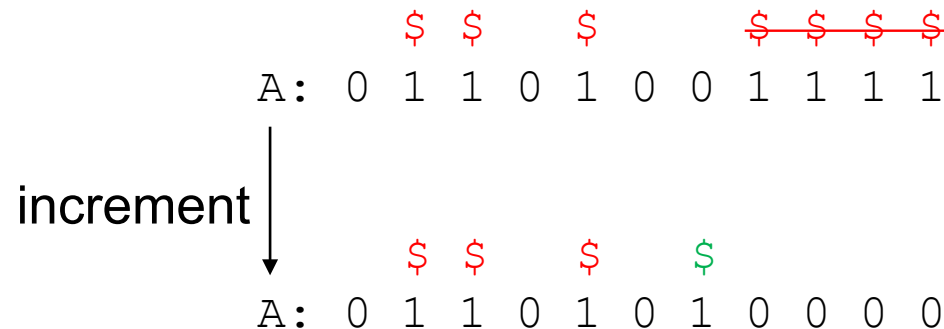
- The least significant bit flips in every increment.
- The second least significant bit flips in $\frac{1}{2}$ of all increments.
- The third least significant bit flips in $\frac{1}{4}$ of all increments.

$x = 000000000000000000000000$

$x = 000000000000000000000001$

$x = 000000000000000000000010$

$x = 000000000000000000000011$



Accounting: 1s carry credit to pay to switch themselves back to 0s

Potential: $\Phi = \#$ of 1s in counter.
The amortized cost of an increment (say, switching b 1s to 0s) is
actual cost + $(\Delta\Phi)$
 $b+1 + (-b+1) = 2$

Recursive Slowdown

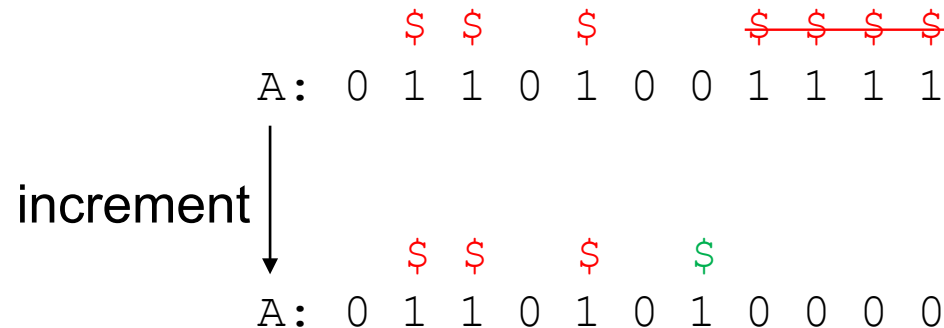
- The least significant bit flips in every increment.
- ... and leads to $\frac{1}{2}$ a unit of work in the second least significant pos.
- ... and leads to $\frac{1}{4}$ a unit of work in the third least significant pos.

x = 000000000000000000000000

x = 000000000000000000000001

x = 000000000000000000000010

x = 000000000000000000000011

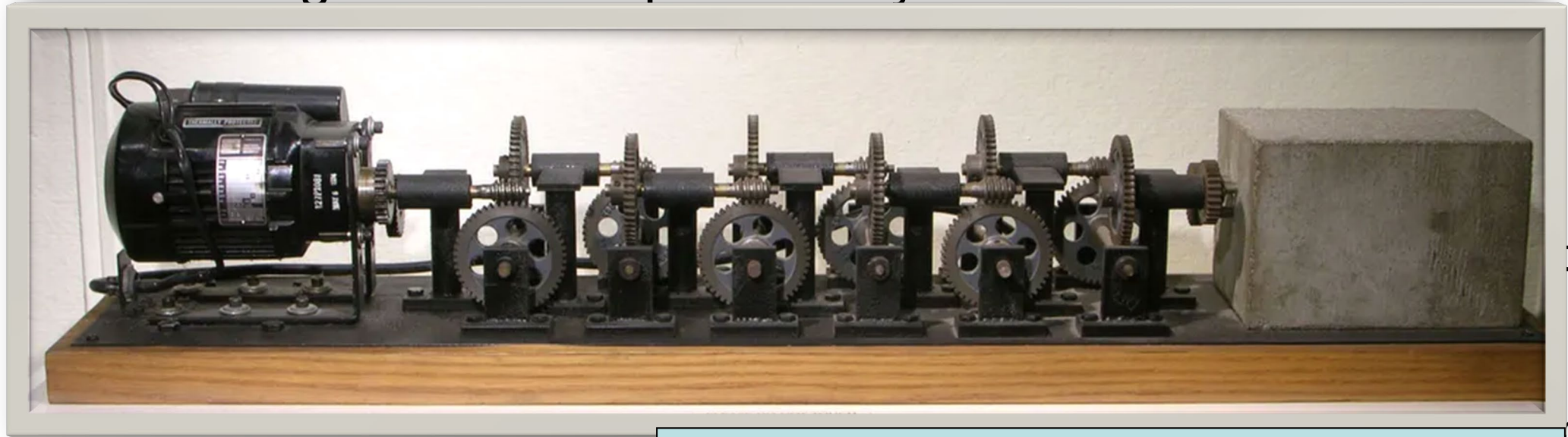


Accounting: 1s carry credit to pay to switch themselves back to 0s

Potential: Φ = # of 1s in counter.
The amortized cost of an increment (say, switching b 1s to 0s) is
actual cost + $(\Delta\Phi)$
 $b+1 + (-b+1) = 2$

Recursive Slowdown

- The least significant bit flips in every increment.



Arthur Ganson, "Machine With Concrete", 1992

increment
 ↓
 A: 0 1 1 0 1 0 0 1 1 1 1
 A: 0 1 1 0 1 0 1 0 0 0 0

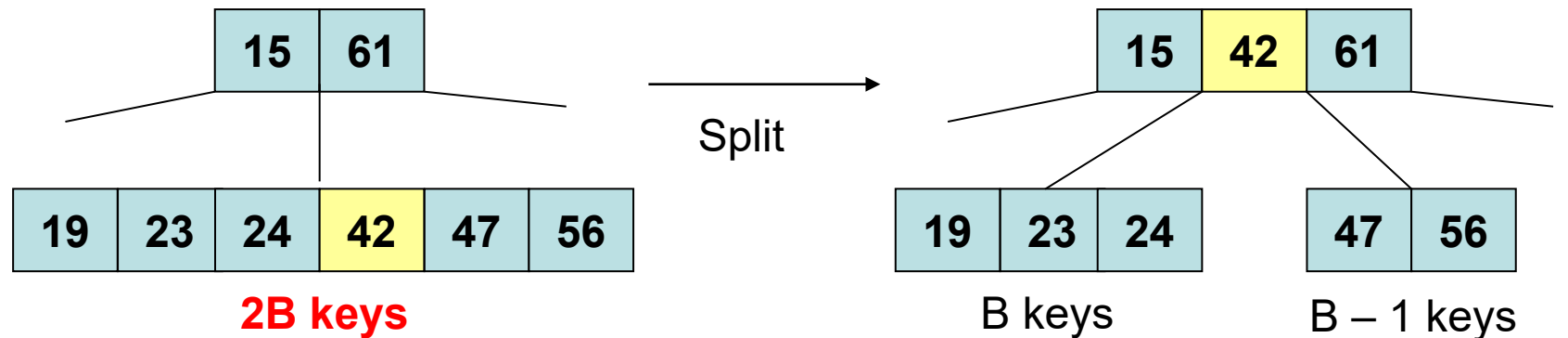
(say, switching b 1s to 0s) is
 actual cost + $(\Delta\Phi)$
 $b+1 + (-b+1) = 2$

to pay
 0s

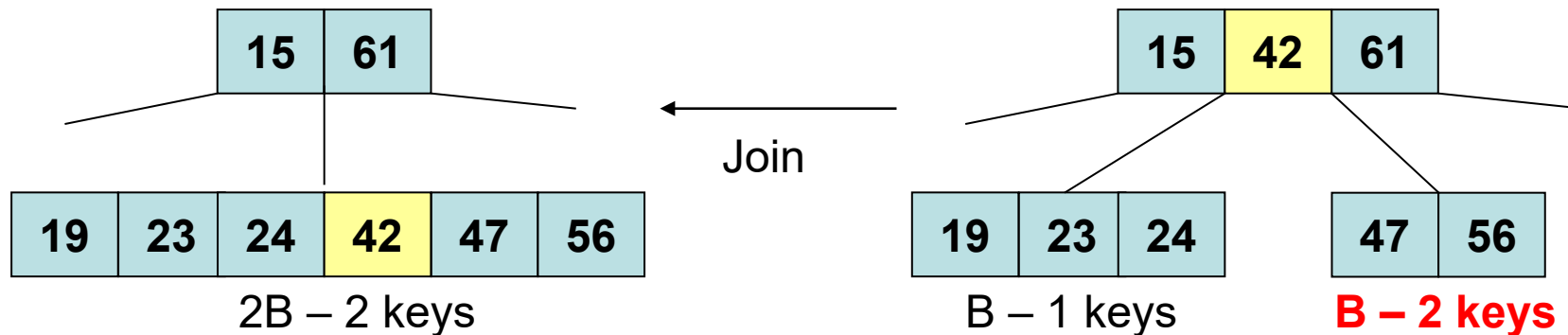
inter.
 increment

B-Tree Insert and Delete

- Insertion might cause a cascade of splits from a leaf upward.
- Deletion might cause a cascade of joins from a leaf upward.



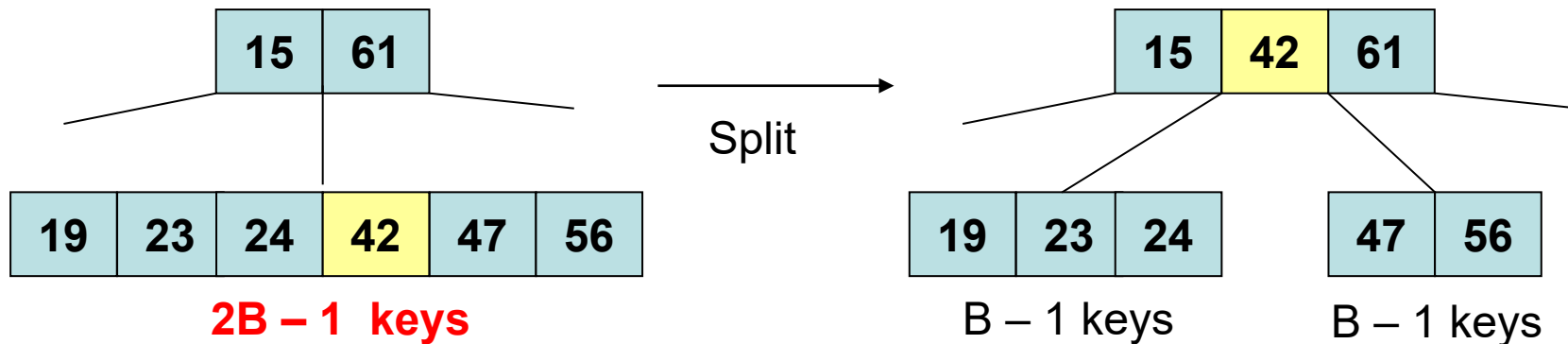
Valid key range:
 $B - 1 \dots 2B - 1$



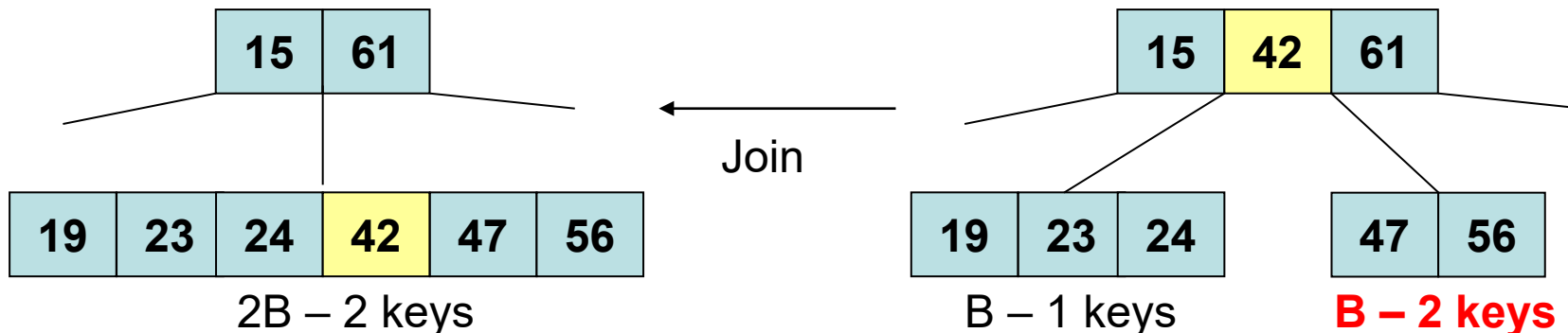
B-Tree Insert and Delete

Why Not Use Tighter Ranges?

- Insertion might cause a cascade of splits from a leaf upward.
- Deletion might cause a cascade of joins from a leaf upward.



Valid key range:
 $B - 1 \dots 2B - 2$
(was $B - 1 \dots 2B - 1$)



Recursive Slowdown Again...

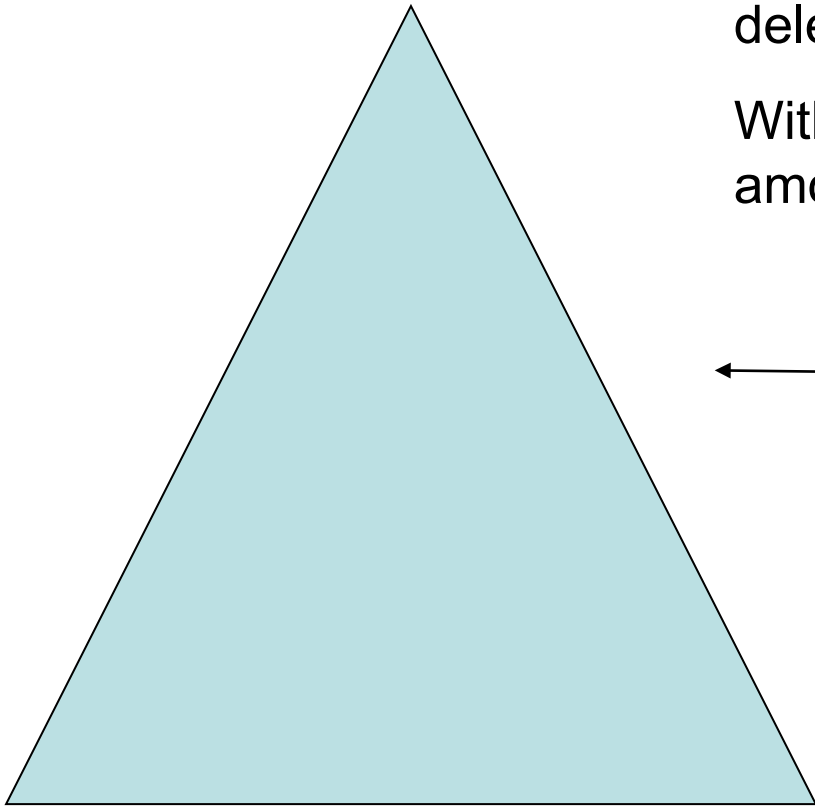
The added slack in our bounds makes insert and delete cause only $O(1)$ amortized splits and joins.

With an appropriate value of B , this means $O(1)$ amortized memory writes per insert / delete!

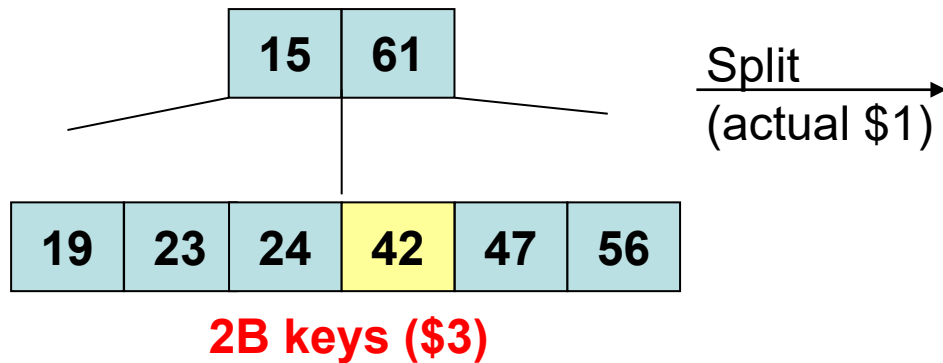
← Consecutive splits / joins need ≥ 8 inserts / deletes
(must have ≥ 2 split / join operations below)

← Consecutive splits / joins need ≥ 4 inserts / deletes
(must have ≥ 2 split / join operations below)

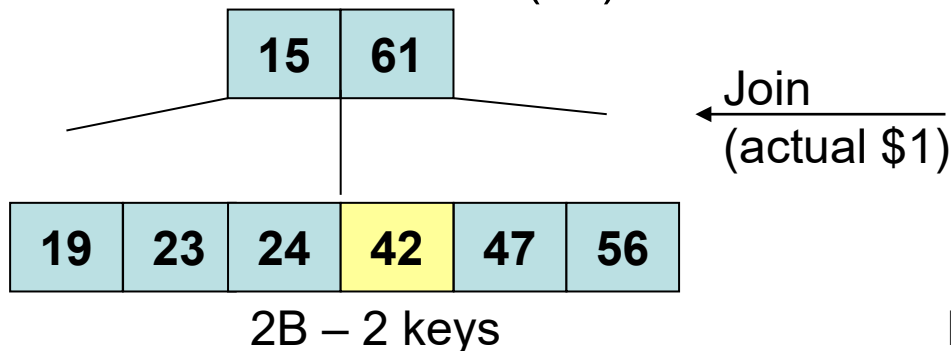
← Consecutive splits / joins need ≥ 2 inserts / deletes
(at least 1 intervening “fast” insert or delete)



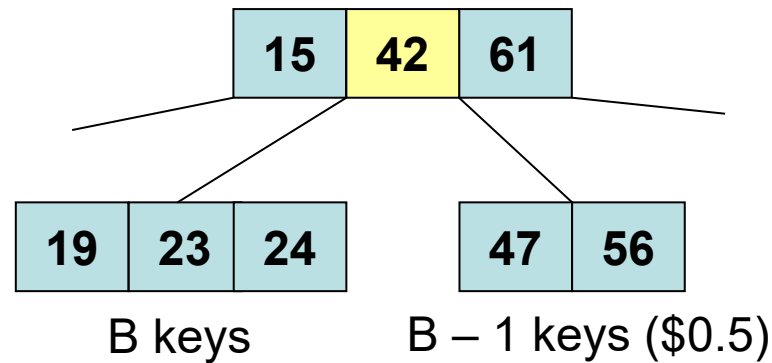
B-Tree Insert and Delete: $O(1)$ Amortized Block Writes



Needs +\$0.5 since possibly:
Reaches lower limit (\$0.5) or
Becomes underfull (\$1)



Needs +\$1.5 since possibly:
Reaches upper limit (\$1.5) or
Becomes overfull (\$3)

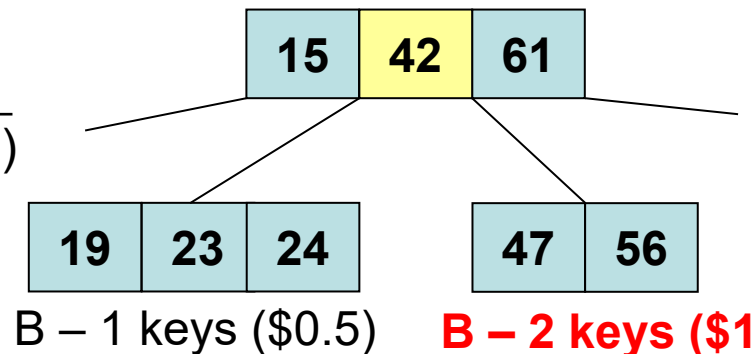


Valid key range:
 $B - 1 \dots 2B - 1$

Insert / Delete now
involve $O(1)$ am splits
and joins!

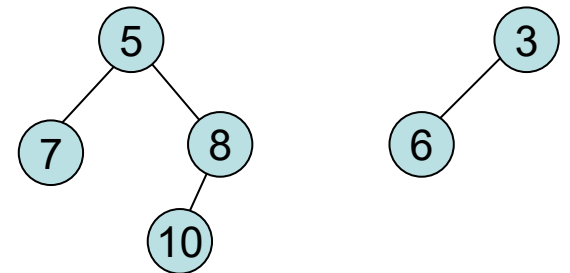
Potential:

$\Phi = 3$ (# overfull nodes)
+ 1.5 (# max nodes)
+ 1 (# underfull nodes)
+ 0.5 (# min nodes)



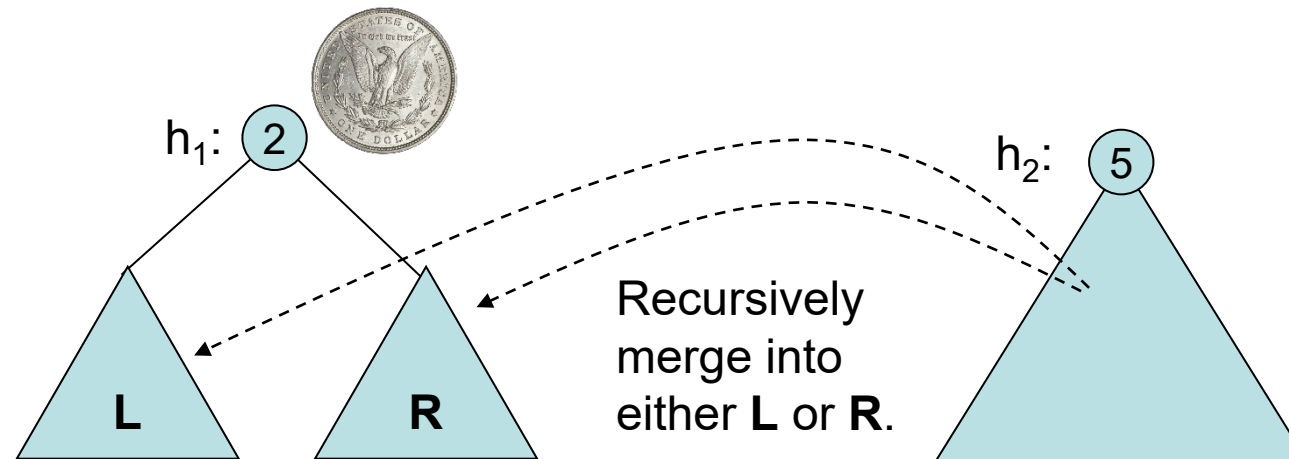
Back to Priority Queues and Mergeable Heaps

- As with randomly mergeable binary heap, store elements in heap-ordered binary trees.
- Suppose we can *merge* two trees in $O(\log n)$ time.
- All priority queue operations now easy to implement in $O(\log n)$ time!
 - *insert*: merge with a new 1-element tree.
 - *remove-min*: remove root, merge left & right subtrees.



Merging Two Heap-Ordered Trees: Hopefully a Familiar Picture

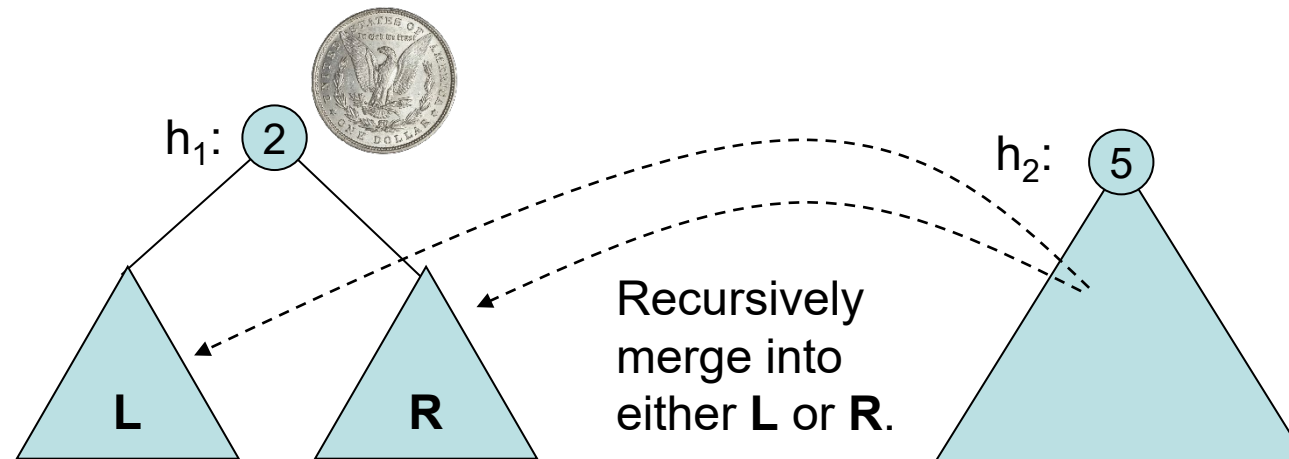
- Take two heap-ordered trees h_1 and h_2 , where h_1 has the smaller root.
- Clearly, h_1 's root must become the root of the merged tree.
- To complete the merge, recursively merge h_2 into either the left or right subtree of h_1 :



- **Randomly mergeable binary heap:**
Decide between these cases with fair coin on h_1 's root that we flip.

The Skew Heap

- Take two heap-ordered trees h_1 and h_2 , where h_1 has the smaller root.
- Clearly, h_1 's root must become the root of the merged tree.
- To complete the merge, recursively merge h_2 into either the left or right subtree of h_1 :

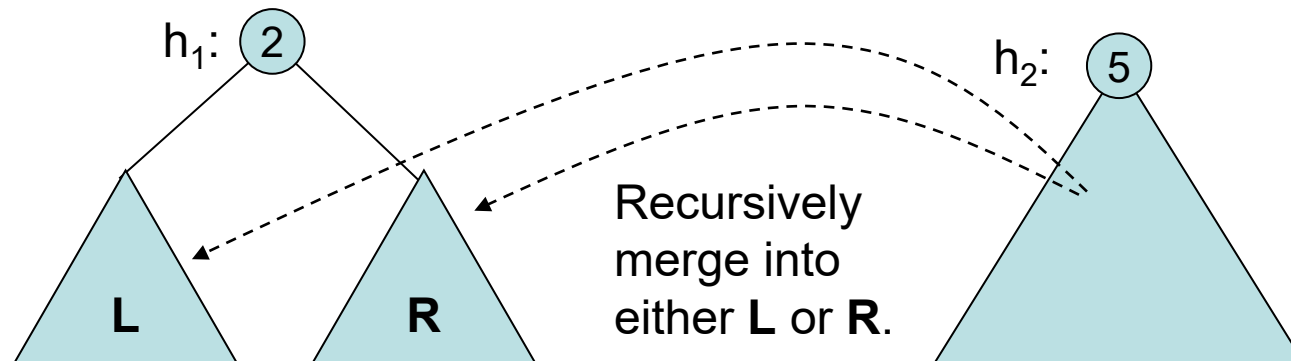


- **Skew Heap:**

Decide between these cases with fair coin on h_1 's root that we flip **OVER**.

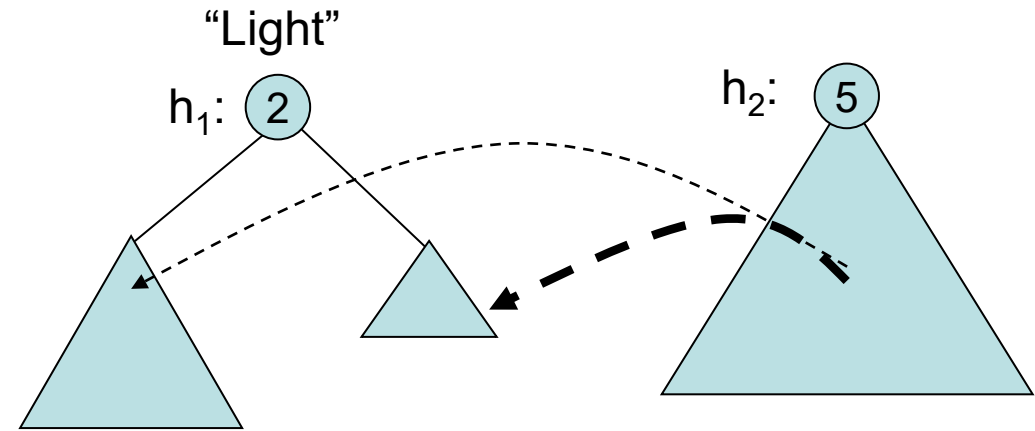
Skew Heaps

- Alternate between merging into L and R.
- Equivalent (no coin needed!): Always merge into R, but after merging h_2 into h_1 , just swap h_1 's children.
- Remarkably, this makes merge run in just $O(\log n)$ amortized time!
- So insert and remove-min also run in $O(\log n)$ amortized time...



Skew Heaps: Amortized Analysis

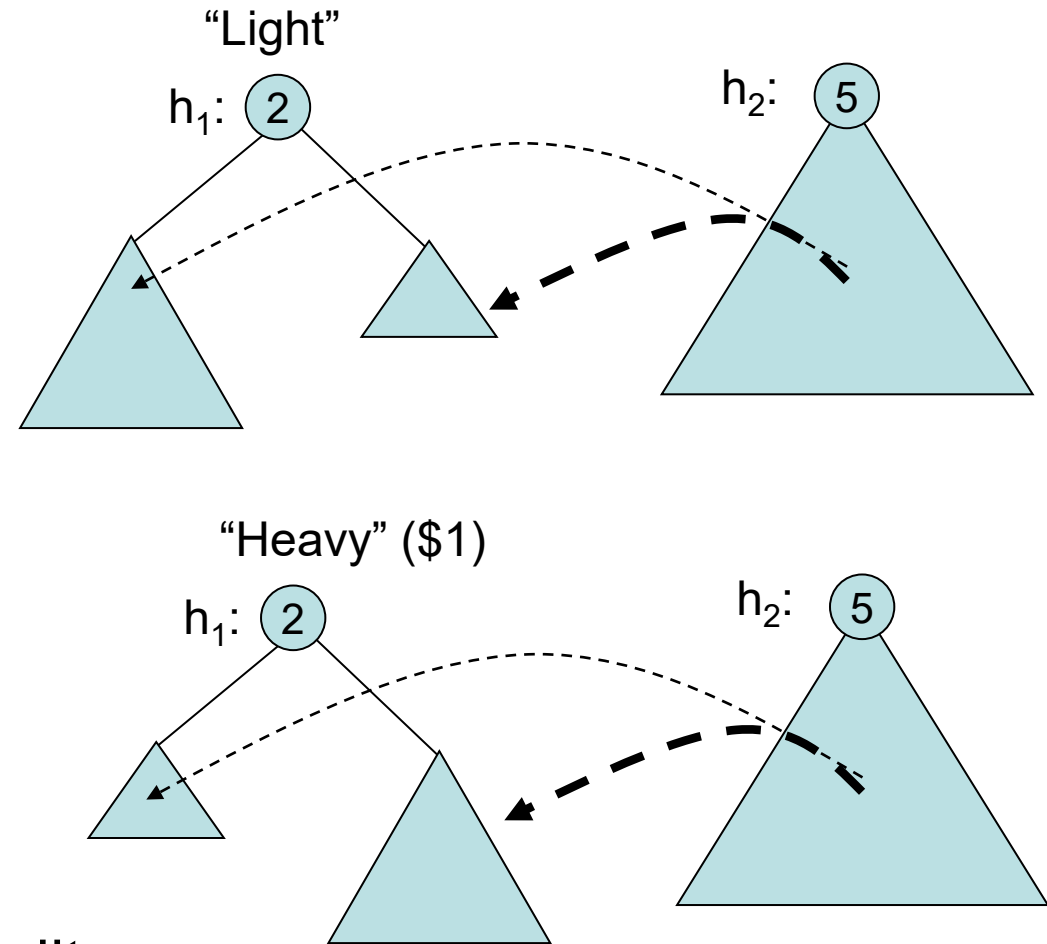
- Call nodes as “heavy” or “light”.
 - Light: right subtree smaller than left
 - Heavy: right subtree larger than left
- Let L and H be the number of steps in a merge that involve light and heavy nodes.
- Easy to argue that $L \leq 2 \log n$
(since h_1 reducing to $\leq \frac{1}{2}$ its size)



A step of the merge involving a light node

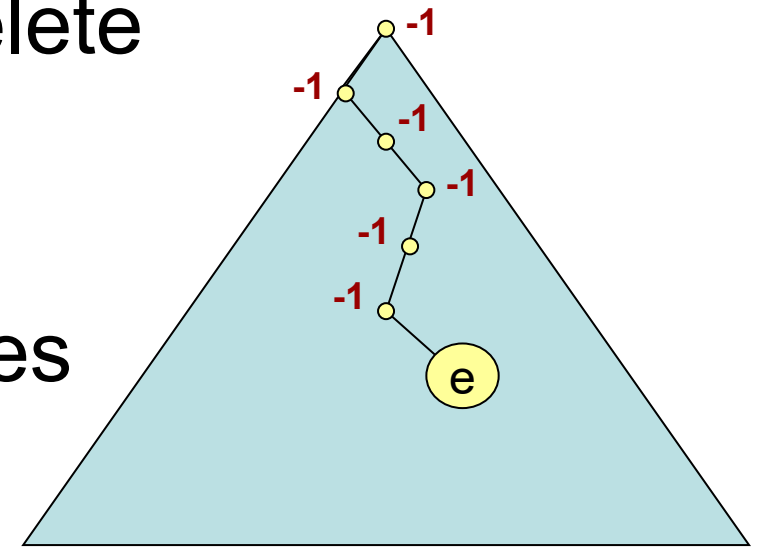
Skew Heaps: Amortized Analysis

- Call nodes “heavy” or “light”.
- Let L and H be the number of steps in a merge that involve light and heavy nodes.
- Easy to argue that $L \leq 2 \log n$ (since h_1 reducing to $\leq \frac{1}{2}$ its size)
- H could be large $\rightarrow$ pay for with \$1 credit on each heavy node.
 - Heavy nodes participating in merge conveniently become light
 - Light nodes might become heavy: need to add $\leq \$2 \log n$ additional credit.



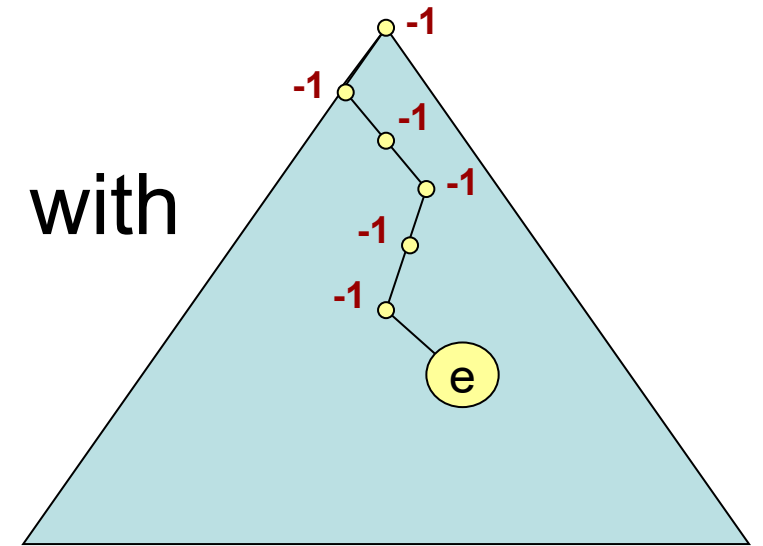
What About Delete?

- In a skew heap, *insert* and *remove-min* are based on merging, so they run in $O(\log n)$ amortized time.
- For *decrease-key* (and *increase-key*), delete an element and re-insert with new key.
- Concern: will deleting an element deep within the tree possibly cause many nodes to turn heavy? (if so, this would require installing credit on these nodes, making the amortized runtime of delete large)



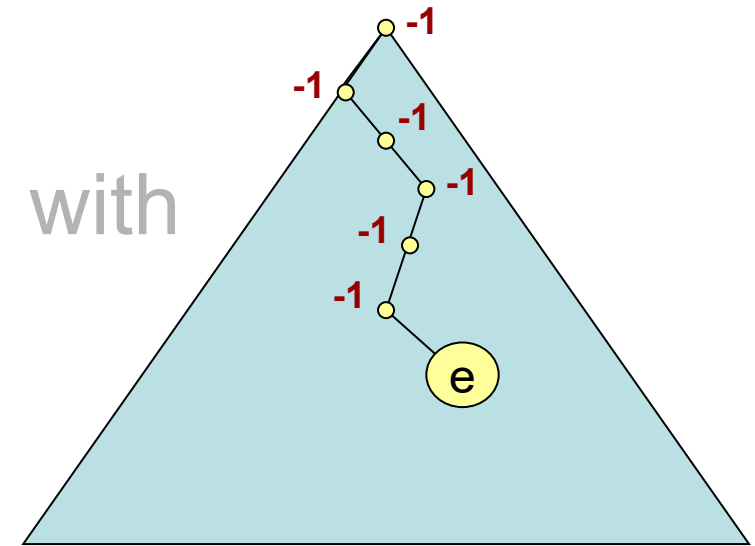
Lazy Deletion

- Don't delete an element, but just mark it as being “deactivated”.
- If remove-min pulls it out later, just call remove-min again.
- Leave $\$ \log n$ credit on element when deactivated to pay for this.
- Problem: data structure may get clogged with deactivated elements, wasting memory.
- If $\geq \frac{1}{2}$ elements in structure deactivated, run $O(n)$ “garbage collect”; rebuild clean copy without deactivated elements. ($\$2$ more credit needed)



Lazy Deletion

- What value of n ?



Other Prominent Examples of Amortization

- Splay trees: $\log(\text{subtree size})$ credit / node.
- Batch tree rebalancing mechanisms
 - Increment counter any time a node is touched during an insert or delete operation. (and add \$3)
 - If counter reaches constant fraction (e.g., $1/3$) of subtree size, rebuild subtree so it becomes balanced.
 - Useful for balancing complicated trees like kd-trees.
- Disjoint sets
- Persistence (enrichment lecture)

Randomization Versus Amortization

- These cases aren't too different:
 - Rebuild data structure in $O(n)$ time every n steps
 - Flip coin and rebuild data structure in $O(n)$ time with probability $1/n$ after each step.
- The first involves $O(1)$ amortized time, the second $O(1)$ expected time.